

コンパイラ演習 2026

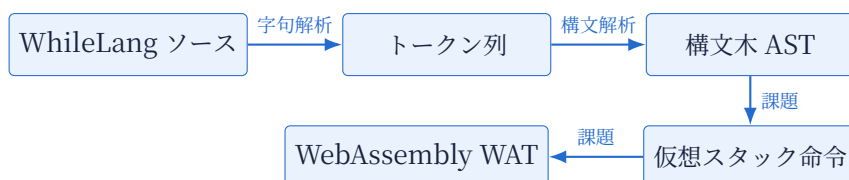
WhileLang から WebAssembly への翻訳

情報システム実験 I 集中講義用レジュメ

この演習では、WhileLang を WebAssembly テキスト形式 (WAT、.wat) へ翻訳する。構文木 (AST) はプログラムを式や文の部品に分けたもの。字句解析と構文解析は配布済みで、構文木から仮想スタック命令への翻訳と WAT の出力を完成させる。

1 コンパイラの見取り図

コンパイラは、実行結果を保ちながらプログラムの表現を変える。今回の翻訳は次の順に進む。青い「課題」の部分を実装する。



実装するファイルは、選んだ言語の次の 2 つである。

- Python: `virtual_stack.py` と `emit_wasm.py`
- C: `virtual_stack.c` と `emit_wasm.c`

最初に Python または C の一方を選んだら、途中で実装を混ぜない。

2 仮想スタック機械

仮想スタック命令は、値を積み、演算し、変数に書き込む。二項演算は「左辺、右辺、演算」の順に出す。次の例では、`a` の値が 3 なら実行後は 6 になる。

WhileLang

```
a := a * 2;
```

仮想スタック命令

```
rvalue a
push 2
*
lpush a
```

重要な区別

- `rvalue a` は変数 `a` の値を積む。
- `lpush a` はスタック先頭の値を変数 `a` に格納する。
- `-` では、下にある値を左辺、上にある値を右辺として計算する。

例: `print 10 - 3;` `push 10` の後は `[10]`、`push 3` の後は `[10, 3]`、`-` の後は `[7]` となり、`print` が 7 を出力する。スタックの右端が先頭である。

仮想命令と WAT の対応

Wasm もスタック機械である。`i32` は 32 ビット整数を表す。仮想命令と WAT の対応を確認しておこう。

役割	仮想命令	WAT
定数を積む	<code>push n</code>	<code>i32.const n</code>
変数を読む	<code>rvalue x</code>	<code>global.get \$x</code>
変数へ書く	<code>lpush x</code>	<code>global.set \$x</code>
加算	<code>+</code>	<code>i32.add</code>
乗算	<code>*</code>	<code>i32.mul</code>
符号付き比較	<code><</code>	<code>i32.lt_s</code>
出力	<code>print</code>	<code>call \$print</code>

3 課題と実装のヒント

3.1 課題 1 算術演算

実装 `Sub` / `Mul` / `Div` の構文木を仮想命令へ翻訳し、`Minus` / `Times` から `i32.sub` / `i32.mul` を出力する。割り算の WAT 出力 `i32.div_s` は配布済みである。

```
case Add(lhs, rhs):
    return compile_arith(lhs) + compile_arith(rhs) + [Plus()]
```

手がかり `Add` と同じ順序で左右の式を翻訳し、最後の命令を変える。C では `ARITH_ADD` と `bin_arith` を手本にする。

確認 `print 3 - 10;` の結果が `-7` になるか調べる。左右を逆に積むと結果も変わる。

3.2 課題 2 比較演算

実装 `GT` / `GE` / `LE` / `EQ` を仮想命令へ翻訳し、対応する WAT を出力する。完成している `LT` を手本にする。

比較	WAT	比較	WAT
<code>></code>	<code>i32.gt_s</code>	<code>>=</code>	<code>i32.ge_s</code>
<code><=</code>	<code>i32.le_s</code>	<code>==</code>	<code>i32.eq</code>

確認 比較結果は真なら 1、偽なら 0。 `print i > 1;` のように比較式を直接出力して調べる。

3.3 課題 3 while

while 文には、条件を調べる場所 **test** と、ループを抜けた後の場所 **out** が必要である。

```
while i < 10 do
  i := i + 1;
```

```
label    L.0          # test
rvalue i
push     10
<
gofalse L.1          # out
rvalue i
push     1
+
lpush    i
goto     L.0
label    L.1
```

命令の順序 LabelTest → 条件 → GoFalse → 本体 → GoTo → LabelOut。GoFalse は条件が 0 なら out へ進み、GoTo は test へ戻る。各 while に対して `gen_label()` で新しいラベルを作ると、入れ子にも対応できる。

WAT では **block** が脱出先を、内側の **loop** が反復先を表す。ここを出力する処理は配布済みである。

```
(block $L.1
  (loop $L.0
    ... condition ...
    i32.eqz
    br_if $L.1
    ... body ...
    br $L.0
  )
)
```

よくある失敗

- GoFalse と GoTo を逆にして、一度も回らない、または無限ループになる。
- LabelTest(test, out) と LabelOut(test, out) に別の組を渡す。
- while の本体が複数文なのに、入力側で `begin ... end` を使わない。

4 デバッグと確認手順

不具合は、翻訳パイプラインの手前から順に調べる。--ast は翻訳が未実装でも表示できる。AST が期待どおりなら翻訳側を調べる。whilei は仮想命令を実行し、whilec は WAT を出力する。

1. 構文木を見る。

```
python3 whilec.py --ast ../test/arith.while
```

2. 仮想命令列を見る。

```
python3 whilec.py --stack ../test/arith.while
```

3. 一命令ずつスタックと変数を追う。

```
python3 whilei.py --trace ../test/arith.while
```

4. 仮想命令が正しく動いたら、WAT を出力して演算命令を確認する。

```
python3 whilec.py ../test/arith.while
```

5. 進捗テストを実行する。

```
python3 test_whilelang.py
```

C 版では ./whilec、./whilei に読み替え、進捗テストは `make test` で実行する。Windows では Python を `py -3` で起動できる。--trace の `stack=` は各命令を実行する直前の状態で、右端が先頭である。発展課題の `if` が未実装なら、必須課題が完了していてもテストに `TODO` が残る。

入力	期待する出力	確認する内容
assign.while	3	配布状態と加算
arith.while	7 8 4 23	減算、乗算、除算、優先順位
cmp.while	1 1 0 1 1 0	比較結果
simple_loop.while	10	基本の while
fact.while	120	while と乗算
loop.while	102 個、最後は 10 10	ラベルの組

5 着手順と発展課題

最初に配布状態で `assign.while` が 3 を出すことを確かめる。その後、課題 1、課題 2、課題 3 の順に進み、短いテストを通してから次へ移る。詰まったときはリポジトリ直下の `HINTS.md` を上から読む。

時間が余ったら、`if P then S else S` を実装する。命令の順序は、条件、`IfStart`、`then` 節、`ElseOp`、`else` 節、`IfEnd` である。Python の命令名は `IfStart` / `ElseOp` / `IfEnd`、C では `I_IF_START` / `I_ELSE` / `I_IF_END` である。